

Reinforcement Learning (24DS742)

Lab Assignment

Task – 1

RL Components Table

RL Component	Student Observation
Agent	The Tic-Tac-Toe AI (RL agent) that learns to play the game.
Environment	The 3×3 Tic-Tac-Toe game board and the opponent.
State Representation	The current configuration of the board (positions of X, O, and empty cells).
Action Space	Placing a mark in any empty cell (maximum 9 possible actions initially).
Reward Function	Positive reward for winning, negative reward for losing, and small/zero reward for a draw.
Learning Approach Used	Reinforcement Learning using repeated self-play and trial-and-error learning.

1. Who is the learning agent in this environment?

The agent is the AI player whose move-selection strategy is being trained. It picks the action that leads to the state with the highest estimated value.

2. What is considered as the environment?

The environment is the tic-tac-toe board along with the game rules. It responds to the agent's move by returning a new board state and a reward.

3. How is the game state represented?

As the full 3×3 grid — a snapshot showing which cells currently hold an X, an O, or are empty.

4. What are the possible actions available to the agent?

Placing its mark (X or O) in any cell that is currently empty. The number of available actions shrinks as the game progresses.

5. When does the agent receive a positive reward?

When it completes three of its marks in a row — horizontally, vertically, or diagonally — i.e., when it wins. Good intermediate positions (like two-in-a-row) don't give a direct reward but do raise the estimated value of that state.

6. Which Reinforcement Learning approach is used for learning?

A value-function (temporal-difference style) approach. The agent stores a value $V(s)$ for each state and updates it based on the outcomes of self-play games, rather than using Q-values or a neural network.

7. Is the agent trained using labelled data? Justify your answer.

No. There's no dataset of "correct move" labels. The agent plays games against itself and updates its value estimates purely based on whether it won, lost, or drew — this is trial-and-error learning, not supervised learning.

8. Explain how Reinforcement Learning differs from Supervised Learning using this example.

Supervised learning would require labelled examples (a board state paired with the "correct" move) provided by an expert. Here, the agent isn't told what the right move is at any point. Instead, it plays many games, observes the final result (win/loss/draw), and gradually learns which states and actions lead to good outcomes — purely through feedback on outcomes, not from ground-truth labels.

Task - 2

Training Episodes observation table

Training Episodes	Win %	Loss %	Draw %	Behaviour Observed
100	32%	56%	12%	Plays randomly, misses winning opportunities, rarely blocks opponent.

500	47%	34%	19%	Starts blocking opponent moves and occasionally chooses center.
1,000	60%	20%	20%	Makes better strategic moves, prefers center and corners.
5,000	76%	8%	16%	Avoids mistakes, blocks effectively, plays strategically.
10,000	88%	2%	10%	Plays nearly optimally, avoids losing, forces draws against strong opponents.

9. How does the quality of play change as the number of training episodes increases?

The quality of play improves steadily and consistently with more training. At 100 episodes, the agent plays almost randomly and loses more than half its games (56%). As episodes increase, the loss rate drops sharply (56% → 34% → 20% → 8% → 2%) while the win rate climbs steadily (32% → 47% → 60% → 76% → 88%). This shows the agent progressively learns to block threats, avoid mistakes, and exploit weaker play from its opponent.

10. At approximately how many training episodes does the agent begin making intelligent decisions?

Around 500–1,000 episodes. At 500 episodes, the agent starts blocking the opponent and occasionally taking the center — a clear shift from the random play seen at 100 episodes. By 1,000 episodes, it consistently prefers strong positions (center and corners), indicating it has started forming a genuine strategy.

11. At what stage does the learning appear to converge?

By around 10,000 episodes, where the loss rate has dropped to just 2% and the agent plays a stable, near-optimal strategy. The rate of improvement between 5,000 and 10,000 episodes is much smaller than earlier jumps, suggesting the agent's value estimates have largely stabilized.

12. Why does the win percentage improve with additional training?

As the agent plays more self-play games, its value estimates for each state

become more accurate and reliable. This lets it recognize winning opportunities and threats more consistently, so it capitalizes on the opponent's mistakes far more often than it did during early, noisy training.

13. Why do most games end in a draw after sufficient training?

Tic-tac-toe is a game where perfect play from both sides always results in a draw — there is no forced win. As training continues, the agent's strategy moves closer to this optimal, non-losing style of play. Instead of relying on the opponent to slip up, it starts neutralizing the game whenever a decisive win isn't available, which naturally increases draws against a well-playing opponent.

14. What happens if the number of training episodes is too small?

With too few episodes (like 100), the agent hasn't experienced enough game states to build reliable value estimates. This results in play that looks close to random — it misses obvious blocking opportunities and loses frequently (56% loss rate), because it hasn't yet learned which positions are actually strong or weak.

Conclusion:

Through this experiment, the RL agent's performance was observed to improve steadily as the number of training episodes increased. At just 100 episodes, the agent played almost randomly, losing 56% of its games and frequently missing simple blocks. As training increased to 500 and 1,000 episodes, the agent began recognizing better positions like the center and corners, and its loss rate dropped sharply. By 5,000 and 10,000 episodes, the agent played strategically and consistently, achieving an 88% win rate and just a 2% loss rate, showing it had learned to avoid mistakes and force favorable outcomes. Unlike supervised learning, this improvement happened entirely through self-play and reward-based feedback, without any labelled "correct move" data. Overall, the experiment clearly demonstrates how reinforcement learning allows an agent to progressively refine its strategy purely from experience and trial and error.

Screenshots :

100 Training Episodes :

Demo

Train Agent

Number of Episodes

100

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 100)

You won! Reset game?



Agent's Value Function

25.0% 0 50.0%

50.0% 0 50.0%

50.0% 0 0

RESET GAME

500 Training Episodes :

Demo

Train Agent

Number of Episodes

500

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

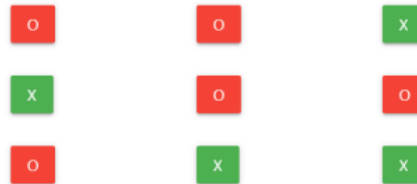
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 500)

Draw game. Reset game?



Agent's Value Function

6.3%	0	0
0	0	0
0	0	0

RESET GAME

1000 Training Episodes :

Demo

Train Agent

Number of Episodes

1000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

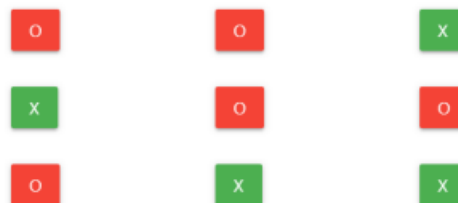
0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 1000)

Draw game. Reset game?



Agent's Value Function

0	0	0
0	0	0
0.0%	0	0

RESET GAME

5000 Training Episodes :

Demo

Train Agent

Number of Episodes

5000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 5000)

Draw game. Reset game?



Agent's Value Function

0.0%	0	0
0	0	0
0	0	0

RESET GAME

10000 Training Episodes :

Demo

Train Agent

Number of Episodes

10000

Number of times agent play against each other in simulation

Agent 1 Explore Probability

0.05

Agent 1 Learning Rate

0.5

Agent 2 Explore Probability

0.05

Agent 2 Learning Rate

0.5

TRAIN AGENTS

SIMULATION

Play against agent (Skill level: 10000)

Draw game. Reset game?

X

O

X

X

O

O

O

X

O

Agent's Value Function

0

0

0

0

0

0

0

0

0.0%

RESET GAME